15,471,856 members Sign in

CODE PROJECT For those who code

articles Q&A forums stuff lounge ? Search for articles, questions,

Understanding Windows Message Queues for the C# Developer



honey the codewitch

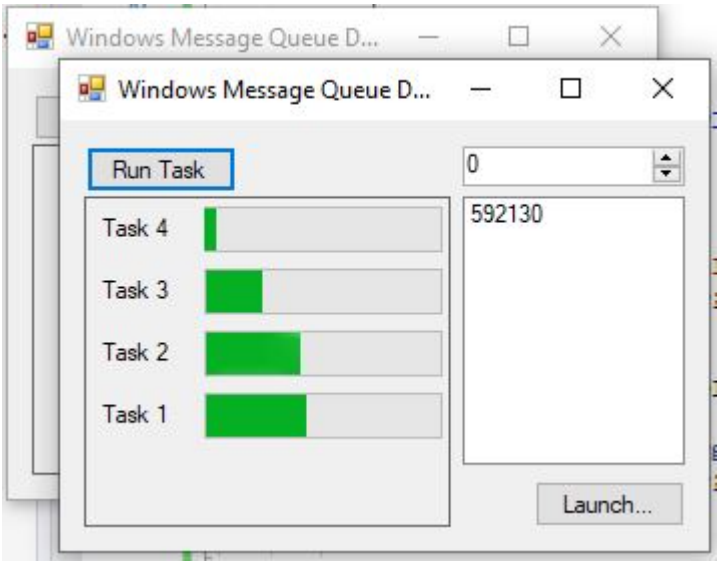
22 Jul 2020 MIT

Rate me: 4.77/5 (20 votes)

Diving into some of the core plumbing behind the Windows operating system

In this article, we explore the Windows message queue system and use it to communicate across threads and even processes. Furthermore, we dive into some of what happens when you call Application.Run(). This article should give you a good understanding of the Windows message queue subsystem.

Download source files - 16.3 KB



Introduction

This article explores the mechanics behind the messaging system in the Windows family of operating systems. It's not so much about giving you a new coding toy to work with as it is about explaining some of the fundamentals of how Windows works, with an eye toward an audience of C# developers. Understanding Windows better in turn, can help you become an even better coder.

Conceptualizing this Mess

A common way to pass information between different threads or even different processes involves message passing. This is where one thread or process can post a message to be received by another thread or process in a safe manner. Windows uses messages to notify its windows of events like painting, resizing, typing and clicking. Each time one of those actions happens, a message is sent to the window, notifying it of that event, at which point it can process it. Underneath the covers of the `Form.MouseMove` event for example, it is being raised in response to a `WM_MOUSEMOVE` message from Windows.

That makes sense for user interface stuff, but we can also use message passing with invisible windows, and define our own messages to post to the window. In this way, we can use one of these invisible windows as a message sink. It gets called whenever it receives a message, and we can hook that. The advantage of using a window for this is that posting and receiving messages to and from windows is thread safe, and works across processes. It should be noted that there are better options for remoting in .NET that don't tie you down to the Windows operating system, but we're exploring this anyway because WinForms, like Windows is built on this.

Just What is a Window?

A window is an object that has an associated message queue and potentially, but not always presents user interface. It has an associated "window class" that tells us the kind of window it is, and we can define our own window classes, which we'll be doing. Furthermore, it has a handle that can be used to refer to it, and the window class has a callback mechanism to notify windows of that class of incoming messages. For one or more windows, there is one thread that is spinning a loop, wherein it is getting messages and then dispatching messages. This thread drives all of the associated windows. Usually, all visible windows are created on the application's main thread. Let's explore this a bit more.

Let's look at a simple C program to register a custom window class and then display a window. This is a Microsoft example for doing exactly that. Don't worry if you don't know C, just follow along as much as you can as I'll be explaining and we'll get to some C# code eventually anyway, I promise:

C++ Shrink ▲

```
HINSTANCE hinst;
HWND hwndMain;

int PASCAL WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance,
    LPSTR lpszCmdLine, int nCmdShow)
```

```
{
    MSG msg;
    BOOL bRet;
    WNDCLASS wc;
    UNREFERENCED_PARAMETER(lpszCmdLine);

    // Register the window class for the main window.

    if (!hPrevInstance)
    {
        wc.style = 0;
        wc.lpfnWndProc = (WNDPROC) WndProc;
        wc.cbClsExtra = 0;
        wc.cbWndExtra = 0;
        wc.hInstance = hInstance;
        wc.hIcon = LoadIcon((HINSTANCE) NULL,
            IDI_APPLICATION);
        wc.hCursor = LoadCursor((HINSTANCE) NULL,
            IDC_ARROW);
        wc.hbrBackground = GetStockObject(WHITE_BRUSH);
        wc.lpszMenuName =  "MainMenu";
        wc.lpszClassName = "MainWndClass";

        if (!RegisterClass(&wc))
            return FALSE;
    }

    hinst = hInstance;  // save instance handle

    // Create the main window.

    hwndMain = CreateWindow("MainWndClass", "Sample",
        WS_OVERLAPPEDWINDOW, CW_USEDEFAULT, CW_USEDEFAULT,
        CW_USEDEFAULT, CW_USEDEFAULT, (HWND) NULL,
        (HMENU) NULL, hinst, (LPVOID) NULL);

    // If the main window cannot be created, terminate
    // the application.

    if (!hwndMain)
        return FALSE;

    // Show the window and paint its contents.

    ShowWindow(hwndMain, nCmdShow);
    UpdateWindow(hwndMain);

    // Start the message loop.

    while( (bRet = GetMessage( &msg, NULL, 0, 0 )) != 0)
    {
        if (bRet == -1)
        {
            // handle the error and possibly exit
        }
        else
        {
```

```
            TranslateMessage(&msg);
            DispatchMessage(&msg);
        }
    }

    // Return the exit code to the system.

    return msg.wParam;
}
```

So there's three or four basic steps here, depending on what you need:

1. Fill a `WNDCLASS` struct with details about the window class, including its name, "MainWndClass", its associated callback procedure, and some styles, which usually don't matter unless the window presents a UI. Once created, register the window class with the OS.
2. Create the window using our registered window class, several style flags, our program's instance handle (`hinst`) and a title "Sample".
3. Show the window and paint it. This only applies if the window is to present a UI. We won't be. It's almost always better to use a `Form` in that case anyway, which wraps all of this.
4. Finally start the message loop. The message loop as I said gets messages and then dispatches messages. Here, we see it translates messages, too which is primarily way for Windows to deal with keyboard "accelerator" shortcuts. Finally, it dispatches messages. This calls the WndProc window's callback function we passed earlier for every message that is received. Without a message loop, a window would be "frozen", as it could not respond to messages so it would never know when to move, or paint or resize, or process user input. Once again, a thread's message loop can drive several windows.

Now What About A Message Loop?

When you call `Application.Run()` in a .NET WinForms application, notice it blocks until your application exits. Inside `Application.Run()`, most likely buried under layers of fluff is one these message loops. It looks something like `while(0!=GetMessage(ref msg)) { TranslateMessage(ref msg); DispatchMessage(ref msg); ... }`, basically like before in the C code. That loop is what's tying up your program on the `Application.Run()` call. Every single `Form` you create will share that same thread's message loop. This thread is called the UI thread. The UI thread is important because it's the one "the user sees" and other threads typically need to communicate with it in order to accept or update information to and from the UI, but they must do so in a thread safe manner. That's why we use message passing, which can post and receive messages safely accross threads. We can communicate from an auxiliary thread back to the UI thread that way, and then the UI thread itself can use that information to update the UI window, which is then a safe operation since it isn't doing anything from the auxiliary thread at that point.

We won't be creating our own message loop in this article, but we'll be doing the rest. We'll also "tap into" the UI thread's message loop by creating another window on that thread, and then receive messages on it. That way, at the end of the day, we will get our code to execute from somewhere inside `Application.Run()`'s loop.

What's a Message Look Like?

A message consists of an `int` message identifier, and two `IntPtr` parameters whose meanings vary depending on what the message identifier is. The good news is, that's all it is. The bad news is, that's all it is. If you need to pass more information than you can wedge into two `IntPtr`s, then you'll have to figure out a way to break it across multiple messages, or allocate from the unmanaged heap and use that to hold the parameters, passing the pointer as one of the `IntPtr`s. That last technique does not work across process.

What Are We Going to Do With Them?

In the demo app, we have two fundamental portions, on the left and then on the right, respectively. On the left is a simple demonstration of the interthread communication. On the right is a simple demonstration of interprocess communication. The left side spawns a task every time you click "**Run Task**", and the task communicates the progress back to the UI thread using the message queue. On the right, you can choose another instance of the app to connect to (if any others exist), launch another instance, and then use the `ListBox` and the provided `NumericUpDown` control to send whichever number you set to the window that is selected in the list box. The other application will then show a message box confirming the message receipt. The `ListBox` only shows window handles, which aren't very friendly, It would be possible to make some sort of friendly application instance numbering system for these but it's a lot of added complexity and would distract from the core goals here.

Coding this Mess

The MessageWindow Class

First up, we must create a `MessageWindow` class which deals with all the window mechanics, including creation, receiving and posting messages, and enumerating other windows. I tried to use `System.Windows.Forms.NativeWindow` for this initially, which probably would have worked except it has no facility for registering a custom window class, and we need that for our interprocess communication. I could have most likely hacked my way around it, but with all the other P/Invoke calls I needed to make against the window anyway it only would have saved me a trivial amount of code, and I didn't want to run into unforseen limitations using it later, since we're going off book with it. Let's look at the constructor, since the code here is similar to part of the C code we saw before:

C# Shrink ▲

```
if (string.IsNullOrEmpty(className))
    className = "MessageWindow";
_wndProc = WndProc;

// Create WNDCLASS
var wndclass = new WNDCLASS();
wndclass.lpszClassName = className;
```

```
wndclass.lpfnWndProc = _wndProc;

var classAtom = RegisterClassW(ref wndclass);

var lastError = Marshal.GetLastWin32Error();

if (classAtom == 0 && lastError != ERROR_CLASS_ALREADY_EXISTS)
{
    throw new Exception("Could not register window class");
}

// Create window
_handle = CreateWindowExW(
    0,
    wndclass.lpszClassName,
    String.Empty,
    0,
    0,
    0,
    0,
    0,
    IntPtr.Zero,
    IntPtr.Zero,
    IntPtr.Zero,
    IntPtr.Zero
);
```

Notice we're registering a `WNDCLASS` and creating the window like we did before in C. There's no message loop this time, though. That's because for this project we're using the message loop inside `Application.Run()`. There's no flag or other setting for that. Basically, Windows "knows" which windows belong to which message loops, based on the thread they were created on so we don't need to specify which one we're using. All we have to do is create the window on the main thread. I want to note that the `MessageWindow` code originally came from MoreChilli over at StackOverflow. Since it was well written, it saved me some time since all I had to do was adapt it for my purposes rather than write it from scratch.

Let's move on to some other important bits of this class. Let's look at our `WndProc` window procedure callback:

C#

```
IntPtr WndProc(IntPtr hWnd, int msg, IntPtr wParam, IntPtr lParam)
{
    if (msg >= WM_USER && msg < WM_USER + 0x8000)
    {
        var args = new MessageReceivedEventArgs(msg - WM_USER, wParam, lParam);
        MessageReceived?.Invoke(this, args);
        if (!args.Handled)
            return DefWindowProcW(hWnd, msg, wParam, lParam);
        return IntPtr.Zero;
    }
    return DefWindowProcW(hWnd, msg, wParam, lParam);
}
```

This routine gets called whenever a message is posted to our message window. Here, we're looking for Windows message ids that are within the range of `WM_USER` (0x400) to `WM_USER+0x7FFF`. If so, we create a new `MessageReceivedEventArgs` with our message information, subtracting `WM_USER` from `msg` to make it zero based, and then we raise the `MessageReceived` event. After the event, we check if any of the event sinks have set the value `Handled` to `true`. If so, we don't call the default window procedure for the window, since we don't want to pass it along for further handling. The reason for this `WM_USER` stuff is because we want to define our own window messages, and messages in the range we accept are the range of "user defined" window messages reserved by Windows. We don't want to handle things like mouse movement messages because this window does not present a user interface. We don't care about the standard messages we could be receiving - just the user defined ones.

Another really important feature of `MessageWindow` is the enumeration of all message windows active on the current machine. This is provided for the IPC demo. We need to see all the available windows so we can present the list to the user, which they can then use to communicate with the foreign `MessageWindow`:

C#

```
public static IReadOnlyList<IntPtr> GetMessageWindowHandlesByClassName(string className)
{
    if (string.IsNullOrEmpty(className))
        className = "MessageWindow";
    var result = new List<IntPtr>();
    var sb = new StringBuilder(256);
    EnumWindows(new EnumWindowsProc((IntPtr hWnd, IntPtr lParam) =>
    {
        GetClassNameW(hWnd, sb, sb.Capacity);
        if (className == sb.ToString())
        {
            result.Add(hWnd);
        }
        return true;
    }), IntPtr.Zero);
    Thread.Sleep(100);

    return result;
}
```

This is a weird routine because of the way `EnumWindows()` works. You'd think an enumeration of windows would have a clear count to it but it doesn't. It calls you back repeatedly with new window handles, never saying when it has given you all of them. Because of this, we have to `Sleep()` for 100ms to give it time to enumerate. This should be more than enough time. I don't like this, but there's not much to be done. Note that we're comparing the window class name of each returned window with the passed in `className` before we add it. You can also pass a `className` to the `MessageWindow` constructor. They have to match. This way, you can create new message windows with different class names, and then get the list of the windows with the class name you want.

`PostMessage()` posts a message to the window in a thread safe manner. `PostRemoteMessage()` posts a message to another window in a thread safe manner. They both use the Win32 `PostMessage()` call

under the covers in order to work:

C#

```
public void PostMessage(int messageId, IntPtr wParam, IntPtr lParam)
{
    PostMessage(_handle, messageId + WM_USER, wParam, lParam);
}
public static void PostRemoteMessage
(IntPtr hWnd, int messageId, IntPtr parameter1, IntPtr parameter2)
{
    PostMessage(hWnd, messageId + WM_USER, parameter1, parameter2);
}
```

The main difference between the two methods is `PostRemoteMethod()` is `static` and takes a window handle to a remote window - actually any window, as Windows doesn't care if it's local to the process or not.

The rest of the code is just P/Invoke definitions and miscellaneous fluff.

The Demo Application

The demo as I implied before, allows you to use `MessageWindow` to receive messages from other threads or processes. The demo code also has facilities for transmitting messages to other processes. It uses `MessageWindow` to do the heavy lifting. First, we create it in the form's constructor, and add the corresponding code to safely destroy it in the `OnClosed()` virtual method:

C#

```
MessageWindow _msgWnd;

public Main()
{
    InitializeComponent();
    _msgWnd = new MessageWindow("MainMessageWindow");
    _msgWnd.MessageReceived += _msgWnd_MessageReceived;
    ProcessListBox.Items.Clear(); // sanity
    foreach (var hWnd in
MessageWindow.GetMessageWindowHandlesByClassName("MainMessageWindow"))
        if(_msgWnd.Handle!=hWnd)
            ProcessListBox.Items.Add(hWnd.ToInt64().ToString());
}

protected override void OnClosed(EventArgs e)
{
    if (null != _msgWnd)
    {
        _msgWnd.Dispose();
        _msgWnd = null;
    }
    base.OnClosed(e);
}
```

Note how we gave the window the window class name of "MainMessageWindow". We also hook `_msgWnd`'s `MessageReceive` event so that we can respond to incoming messages. After that, we clear the list just in case (though it should be clear, if you add stuff to it in the designer it won't be), and enumerate all the windows on the system except our own, populating the `ListBox` with their values.

As mentioned, our `MessageReceived` handler covers responding to the incoming window messages on `_msgWnd`:

C#

```
void _msgWnd_MessageReceived(object sender, MessageReceivedEventArgs args)
{
    switch(args.MessageId)
    {
        case MSG_REMOTE:
            MessageBox.Show("Value is " + args.Parameter1.ToInt32().ToString(),
                            "Remote message received");
            args.Handled = true;
            break;
        case MSG_PROGRESS:
            var ctrl =
                TaskPanel.Controls[args.Parameter1.ToInt32()-1] as WorkerProgressControl;
            if(null!=ctrl)
                ctrl.Value = args.Parameter2.ToInt32();
            args.Handled = true;
            break;
    }
}
```

Here we have two possibilities that we care about: receiving a remote message, and receiving a message on the progress of any currently executing tasks from other threads. In the first possibility, we simply show a message box displaying the first parameter of our message which contains the value specified by the remote process. This will be the value in the remote process' `NumericUpDown` control, which we'll get to.

The second possibility is a message that tells us to update to corresponding task's `ProgressBar`. This message comes from a local thread that represents the task. The first parameter contains the task's id, so we know which progress bar to update. The second parameter is the progress bar value. Since the `MessageReceived` event fires on the UI thread, because it's called from inside `Application.Run()` we are safe to update our `Form`'s controls.

We have two more bits of code to cover, the first being our code to send a remote message, which we do whenever the value of our `NumericUpDown` control's `Value` changes:

C#

```
void TransmitUpDown_ValueChanged(object sender, EventArgs e)
{
    if(-1< ProcessListBox.SelectedIndex)
    {
        MessageWindow.PostRemoteMessage(
```

```
            new IntPtr(int.Parse(ProcessListBox.SelectedItem as string)),
            MSG_REMOTE,
            new IntPtr((int)TransmitUpDown.Value),
            IntPtr.Zero);
    }
}
```

Here, we check if one of the handles in our `ListBox` is selected, and if it is, we post a remote message with the id of `MSG_REMOTE` to the selected handle sending the `Value` as the first message parameter. That will cause the remote process's `MessageReceived` event to fire and the corresponding `MessageBox` we covered earlier to be shown.

The next bit of code we have to cover is the local tasks we can create, which we do whenever the "**Run Task**" `Button` is clicked:

C#

```
void RunTaskButton_Click(object sender, EventArgs e)
{
    var id = TaskPanel.Controls.Count + 1;
    var wpc = new WorkerProgressControl(id);
    TaskPanel.SuspendLayout();
    TaskPanel.Controls.Add(wpc);
    wpc.Dock = DockStyle.Top;
    TaskPanel.ResumeLayout(true);
    Task.Run(() => {
        for (var i = 0; i <= 100; ++i)
        {
            _msgWnd.PostMessage(MSG_PROGRESS, new IntPtr(id), new IntPtr(i));
            Thread.Sleep(50);
        }
    });
}
```

Here, he creates a new `WorkerProgressControl` which just has a `Label`, a `ProgressBar`, and a `Value`. We give at an `id` in the constructor, and then we dock it to the top of our autoscrolling tasks `Panel` to create a quick and dirty tasks list. Finally we `Run()` a `Task` that does some faux "work" which is what that loop and everything inside it does. Note we're reporting progress using `_msgWind.PostMessage()` where the first parameter is our task `id` so we know which `WorkerProgressControl` to update, and the second parameter is the progress value.

Windows uses its window message queues to notify a UI of events like mouse clicks and to create thread safe interaction between tasks or IPC between processes. That was a lot of ground to cover, but hopefully now you have a greater understanding of how this works. That's all she wrote!

Disclaimer

The code in this article is not intended for production use. Under normal circumstances, I don't recommend using the windows message queues in .NET. There are other ways to remote, and do message passing and thread synchronization that are cross platform, more capable, and more in line

with the way we do things in .NET. This was for demonstration purposes to give you a better understanding of Windows.

History

- 22nd July, 2020 - Initial submission

License

This article, along with any associated source code and files, is licensed under The MIT License

Written By

honey the codewitch

United States

Just a shiny lil monster. Casts spells in C++. Mostly harmless.

Comments and Discussions

You must Sign In to use this message board.

Search Comments

First Prev Next

Additional useful Windows message function
svansickle 24-Jul-20 11:26

Re: Additional useful Windows message function
honey the codewitch 24-Jul-20 12:38

Thread
Abbathsin 23-Jul-20 9:48

Re: Thread

honey the codewitch 23-Jul-20 12:24

My vote of 5
Member 13704143 22-Jul-20 23:47

Re: My vote of 5
honey the codewitch 23-Jul-20 12:24

Oh, wow ..in less than 24 hours?
asiwel 22-Jul-20 6:58

Re: Oh, wow ..in less than 24 hours?
honey the codewitch 22-Jul-20 7:04

Refresh 1

General News Suggestion Question Bug Answer Joke Praise Rant Admin

Use Ctrl+Left/Right to switch messages, Ctrl+Up/Down to switch threads, Ctrl+Shift+Left/Right to switch pages.

Permalink
Advertise
Privacy
Cookies
Terms of Use

Layout: fixed | fluid

Article Copyright 2020 by honey the codewitch
Everything else Copyright © CodeProject, 1999-2022

Web04 2.8:2022-10-01:1